ISM3232 — Week 8 Lab

Debugging, AI Literacy & Midterm Review — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 08 · Unit 2 · Python Foundations

Contents

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 8 Lab — Debugging, Tracebacks & AI Literacy
Unit	Unit 2 · Python Foundations
Class length	Full class period (75 minutes)
Format	Live code-along, structured around a documented debug log
Prerequisites	Week 7: functions, modules, `pytest`, type hints (marked Midterm-Eligible; last lab before Week 9's midterm)
Student-facing lab page	Week 8 In-Class Lab — Module 5, "Debugging, Tracebacks, and AI Literacy"
Parts covered	Part 1 (read the traceback) – Part 5 (AI reflection + push) + Stretch (a third bug)
Submission	2 screenshots + `debug_log.md`, GitHub URL, Canvas, completion credit

The lab page's own rule, stated as a heading, is the entire lesson of this lab: **Debug First, Then Ask.** Attempt each bug yourself, using the traceback and `print()`, before touching any AI tool — and document the attempt in `debug_log.md` *before* asking AI anything. Say explicitly to the class: **this is graded on process, not just working code.** A student who pastes an error into an AI tool immediately and gets working code back, with no documented independent attempt, has not actually done this lab, even if their script runs correctly at the end. This is also the last lab before Week 9's midterm — a good moment to fold in brief review of anything from Weeks 5–7 that's felt shaky, without derailing today's own content.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Read a Python traceback from the bottom up, identifying the error type, the file/line, and what that error type means.
2. Use `print()` debugging to inspect a variable's actual runtime value when its expected value isn't obvious from reading the code alone.
3. Diagnose and fix a `KeyError` (a typo'd dictionary key) and a `TypeError` (comparing incompatible types).
4. Apply rubber duck debugging — narrating code out loud, literally, line by line — as a genuine diagnostic technique, not a gimmick.

5. Use AI tools narrowly and after independent effort, and write an honest, specific AI-use disclosure.

3 Before Class — Setup Checklist

- ☐ Rehearse both bugs yourself before class, including reading each traceback aloud the way you'll ask students to — Bug 1 is a `KeyError` from a typo'd dictionary key (`'amout'` instead of `'amount'`); Bug 2 (which only appears *after* Bug 1 is fixed) is a `TypeError` from comparing an `int` to a `str`. Both tracebacks are fully verified in this guide's walkthrough below.
- ☐ Have a genuine rubber duck, or announce that any object on a desk will do, before Part 1 — say this without irony; naming the technique's real source (Hunt & Thomas, *The Pragmatic Programmer*, 1999) helps students take it seriously as a real, well-established practice rather than a classroom bit.
- ☐ Decide and state your AI-tool policy explicitly at the start of Part 5, exactly as in comparable AI-literacy exercises earlier in the semester — which tool(s) are permitted, and what "used AI" means for the purposes of the required disclosure.
- ☐ If your section has genuine midterm-review needs from Weeks 5–7, budget a few minutes at the very end for it — but don't let review time encroach on Part 1–3's debugging work, which is this lab's actual, irreplaceable content.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, the existing `module05_functions` venv from Week 7
- Students: a rubber duck or any patient inanimate object
- Access to an AI assistant for the narrowly-scoped Part 5 step only

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:03	Welcome: "Debug First, Then Ask"	3
0:03–0:18	Part 1 — Set up and read the first traceback	15
0:18–0:30	Part 2 — Fix Bug 1	12
0:30–0:42	Part 3 — Fix Bug 2	12
0:42–0:55	Part 4 — Write tests for the fixed code	13
0:55–1:07	Part 5 — AI reflection and push	12
1:07–1:11	Stretch preview (a third bug)	4
1:11–1:15	Wrap-up, submission checklist	4

Parts 1–3 (the actual debugging work) are given the most protected time of the whole lab — this is deliberate; rushing the traceback-reading and rubber-duck steps to “get to the fix faster” defeats the entire point of the exercise.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:03)

Say to the class:

> *"One rule today, and it's graded seriously: Debug First, Then Ask. You will hit two bugs in a script I give you. Before any AI tool, you read the traceback, you try `print()` debugging, and — genuinely — you explain the suspect code out loud to a rubber duck. Document all of that in `debug_log.md` before you ask an AI anything. I'm not grading whether you fixed the bugs — I'm grading whether you can show me how you got there."*

6.2 Part 1 — Set Up and Read the First Traceback (0:03–0:18, 15 min)

Teaching goal: Copy a genuinely broken script exactly as given, run it, and read the resulting traceback methodically — bottom-up, extracting the error type, location, and meaning — before attempting any fix.

Say to the class:

> *"Type this exactly as shown — bugs included, on purpose. Do not fix anything yet. Run it, and we're going to read the error together, slowly, before touching a single line of code."*

Do:

```
cd ~/ism3232/module05_functions
source .venv/bin/activate
touch week8_buggy.py debug_log.md
```

Type this into `week8_buggy.py` exactly as shown:

```python
# week8_buggy.py  -- three bugs -- do not fix yet

records = [
    {'id': 1, 'name': 'Taylor', 'amout': 1200},
    {'id': 2, 'name': 'Jordan', 'amount': 450},
    {'id': 3, 'name': 'Morgan', 'amount': 3500},
]

def get_total(records):
    total = 0
    for rec in records:
        total += rec['amount']
    return total
```

```
def is_over_limit(amount):
    return amount > '1000'

def format_summary(total, count):
    return f'Total: ${total:.2f} across {count} records'

total = get_total(records)
print(format_summary(total, len(records)))
print(is_over_limit(total))
```

Run it:

```
python3 week8_buggy.py
```

Verified traceback:

```
Traceback (most recent call last):
  File "week8_buggy.py", line 19, in <module>
    total = get_total(records)
            ^^^^^^^^^^^^^^^^^^
  File "week8_buggy.py", line 10, in get_total
    total += rec['amount']
             ~~~^^^^^^^^^^
KeyError: 'amount'
```

Read it together, from the bottom up, exactly as the three questions ask:

1. **"What is the error type on the last line?"** — `KeyError`, specifically `KeyError: 'amount'`.
2. **"Which file and line number is highlighted?"** — `week8_buggy.py`, line 10, inside `get_total()`, at `total += rec['amount']`. Say explicitly: **the traceback shows a whole chain** — line 19 (where `get_total` was *called*) and line 10 (where the actual failure *happened*, inside that function) — both matter, but line 10 is where the bug physically lives.
3. **"What does that error type mean?"** — `KeyError` means: code tried to access a dictionary using a key (`'amount'`) that doesn't exist in that specific dictionary. Say explicitly, and this is the genuinely useful diagnostic insight: **the error doesn't say the key is *always* missing — it says the key is missing from *this specific* dictionary, on *this specific* pass through the loop.** Three records exist; only one is broken; the traceback alone doesn't tell you *which* one — that's what Part 2's `print()` debugging is for.

Now, rubber duck debugging — genuinely, not as a formality:

> *"Explain `get_total()` out loud, to the duck, one line at a time. Not what it* should *do — what it* literally *does. 'This line sets `total` to zero. This line starts a loop over `records`. This line adds `rec['amount']` to `total`.' Say it. What you catch yourself hesitating on, or phrasing*

> *awkwardly, becomes your next two log entries."*

Facilitation note: circulate and genuinely listen for a moment when students narrate `rec['amount']` — a student who's paying attention will often say something like "this adds the amount field… assuming it's called 'amount' in every record" — that hesitation is the actual diagnostic insight arriving, worth reinforcing explicitly if you hear it.

Now, fill in `debug_log.md`'s Bug 1 section, before touching any code:

```
# debug_log.md
# Author: [Your Name]

## Bug 1
Error type: KeyError
File + line: week8_buggy.py, line 10 (inside get_total)
What the error means: code tried to access a dictionary key
  ('amount') that doesn't exist in one of the records
What I told the duck: [student's own words]
What I tried: [to be filled in during Part 2]
How I fixed it: [to be filled in during Part 2]
```

Common student mistakes to watch for:

- Reading the traceback top-down and getting stuck on the "Traceback (most recent call last):" line — same reminder as any prior traceback-reading exercise this semester: that line is boilerplate, always identical, never the actual information; the real content is at the bottom.
- Skipping the rubber-duck step as "silly" and going straight to fixing — actively watch for and gently redirect this; the exercise's grading explicitly values the documented process, and skipping it produces a thinner, less honest `debug_log.md` later.
- Guessing at "what the error means" rather than reasoning it out from the literal words `KeyError: 'amount'` — encourage decomposing the message itself: *Key*Error, about the key `'amount'` specifically.

Check for understanding: "The traceback names line 10, inside `get_total()`. Does that mean line 10 itself is *wrong*, or could the bug actually be somewhere else?" (The bug could be — and in this case, is — somewhere else: line 10's code is *correct* in general; the actual problem is in the *data* being fed into it, back at line 4's typo'd `'amout'` key. This is a genuinely important distinction: **a traceback shows you where the failure was *detected*, not necessarily where the mistake was *made*.**)

6.3 Part 2 — Fix Bug 1 (0:18–0:30, 12 min)

Teaching goal: Use `print()` debugging to inspect the actual data flowing through the loop, spot the typo, and fix it at its source.

Say to the class:

> *"The traceback told us* where *the failure happened, not* why *the data was wrong. Now we look at the data itself, directly, with a debug print."*

Live-code this, adding one line inside `get_total()`:

```
def get_total(records):
    total = 0
    for rec in records:
        print(f'DEBUG: rec = {rec}')   # add this
        total += rec['amount']
    return total
```

Run it again. Verified output (before the crash):

```
DEBUG: rec = {'id': 1, 'name': 'Taylor', 'amout': 1200}
Traceback (most recent call last):
  ...
KeyError: 'amount'
```

Say explicitly, pointing at the printed dictionary directly: "There it is — printed in plain sight. `{'id': 1, 'name': 'Taylor', 'amout': 1200}` — read the key names out loud. `'amout'`, not `'amount'`. A single missing letter, and Python has no way to know it was a typo rather than a deliberately different key — it just faithfully reports that `'amount'` isn't there, because it genuinely isn't, under that exact spelling."

Fix it at the source — correct the typo in the `records` list itself:

```
records = [
    {'id': 1, 'name': 'Taylor', 'amount': 1200},   # fixed: was 'amout'
    {'id': 2, 'name': 'Jordan', 'amount': 450},
    {'id': 3, 'name': 'Morgan', 'amount': 3500},
]
```

Remove the `DEBUG` print line — say explicitly why this matters, echoing Week 7's "no `print()` in business logic" rule: a debug print left behind in shipped code is exactly the kind of stray, unintentional output that rule exists to prevent; once its diagnostic job is done, it comes back out.

Run again to confirm Bug 1 is gone. At this point, a **second, different** error should appear — Bug 2, covered next — say explicitly this is expected, not a sign Bug 1's fix failed: "You'll see a new error now. That's not a mistake — that's Bug 2, which was always there, just hidden behind

Bug 1 crashing first."

Update `debug_log.md`'s Bug 1 section — "What I tried" and "How I fixed it":

```
What I tried: added a DEBUG print inside get_total() to see each
  record's actual contents before the key access that was failing
How I fixed it: corrected the typo 'amout' to 'amount' in the first
  record's dictionary; removed the DEBUG print afterward
```

Common student mistakes to watch for:

- Fixing the typo by changing `rec['amount']` (the *access*) to `rec['amout']` (matching the *typo*) instead of fixing the data — this "works" in the narrow sense that it stops crashing, but it's fixing the wrong side of the mismatch, and would break again the moment any *other* record (which correctly uses `'amount'`) is processed. If a student does this, have them run the script and predict what happens to `Jordan`'s and `Morgan`'s totals (a new, different `KeyError`, since *their* records use `'amount'` correctly) — a good, concrete illustration of why fixing the data source, not the access point, is the right call here.
- Leaving the `DEBUG` print in place after the fix — walk the room checking it's been removed, per the explicit instruction and the Week 7 callback.

Check for understanding: "If all three records had used `'amout'` consistently — the *same* typo everywhere — would the original `KeyError` have looked any different?" (No — it would still fail with the exact same `KeyError: 'amount'`, on the very first record instead of just one specific one; the `print()` debugging step would have been just as necessary to spot it, since the traceback alone never shows the dictionary's actual keys, only names the one it was looking for and failed to find.)

6.4 Part 3 — Fix Bug 2 (0:30–0:42, 12 min)

Teaching goal: A `TypeError` from comparing incompatible types — diagnose it the same disciplined way (read, duck, document) before fixing, reinforcing that the process from Bug 1 generalizes to a genuinely different kind of bug.

Say to the class:

> *"New error, new type entirely. Same process: read it, explain it to the duck, document before fixing."*

Verified traceback, after Bug 1's fix:

```
Total: $5150.00 across 3 records
Traceback (most recent call last):
  File "week8_buggy.py", line 21, in <module>
    print(is_over_limit(total))
          ^^^^^^^^^^^^^^^^^^^^
  File "week8_buggy.py", line 14, in is_over_limit
    return amount > '1000'
           ^^^^^^^^^^^^^^^
TypeError: '>' not supported between instances of 'int' and 'str'
```

Say explicitly, pointing out something worth noticing before diagnosing further: "Notice the *first* line printed correctly — `Total: $5150.00 across 3 records` — that's `get_total()` and `format_summary()` both working now, confirming Bug 1's fix is genuinely solid. The crash only happens on the *next* line, inside a completely different function. This is worth stating explicitly: fixing one bug doesn't mean the whole program is correct — it means that *specific* bug is gone."

Now, rubber duck the comparison itself, per the lab's explicit instruction:

> *"Say out loud, to the duck: what type is `amount` at this point in the program? What type is `'1000'`? Is `>` defined between those two types?"*

Facilitation note: a student narrating this correctly should arrive at something like: "`amount` is `total`, which came from `get_total()` — that returns a number, an int or float. `'1000'` has quotes, so it's a string. Comparing a number to a string with `>` — Python doesn't know how to say a number is 'greater than' a piece of text." That reasoning, said out loud, *is* the fix arriving — reinforce this explicitly if you hear it.

Update `debug_log.md`'s Bug 2 section, before fixing:

```
## Bug 2
Error type: TypeError
File + line: week8_buggy.py, line 14 (inside is_over_limit)
What the error means: Python cannot compare an int and a string
```

```
  with > -- the two sides of the comparison are different types
How I fixed it: [to be filled in below]
```

Now fix it — the hint given directly on the lab page:

```
def is_over_limit(amount):
    return amount > 1000   # fixed: '1000' (string) -> 1000 (int)
```

Run the fully corrected script. Verified final output — matches the lab page's own stated expected result exactly:

```
Total: $5150.00 across 3 records
True
```

Update `debug_log.md`'s Bug 2 section:

```
How I fixed it: removed the quotes around 1000 so is_over_limit
  compares two integers instead of an int and a string
```

Common student mistakes to watch for:

- Converting `amount` to a string instead of converting `'1000'` to an integer (`return str(amount) > '1000'`) — this actually runs without error, since string comparison *is* defined, but compares **alphabetically**, not numerically, which would produce wrong results for many inputs (e.g., `'9' > '1000'` is `True` alphabetically, since `'9'` sorts after `'1'` character-by-character, even though `9 < 1000` numerically) — a genuinely instructive "technically works, but is quietly wrong" fix worth flagging explicitly if a student proposes it, even though it happens not to break on today's specific test input.
- Treating this as the "same bug" as Bug 1 rather than a genuinely different error type and cause — worth stating explicitly why they're different: Bug 1 was about a *missing* piece of data (a wrong key name); Bug 2 is about *mismatched types* being compared — different diagnostic categories, both worth being able to recognize by name.

Check for understanding: "If `is_over_limit` were called with a *float* instead of an int — say, `is_over_limit(1500.50)` — would the fixed version still work correctly?" (Yes — `1500.50 > 1000` compares a float to an int just fine, since Python allows numeric comparisons across `int`/`float` freely; the type mismatch that actually caused Bug 2 was specifically between a number and *text*, not between two different numeric types. A good check that the fix's scope is understood precisely, not over-generalized.)

6.5 Part 4 — Write Tests for the Fixed Code (0:42–0:55, 13 min)

Teaching goal: Five `pytest` tests verifying the now-fixed functions, including a boundary case for `is_over_limit` — direct reinforcement of Week 7's boundary-testing lesson, applied to code that was, minutes ago, genuinely broken.

Say to the class:

> *"Now that both bugs are fixed, let's prove it stays fixed — five tests, including the exact boundary-case discipline from last week."*

Live-code this:

```
touch tests/test_week8.py
code tests/test_week8.py
```

```python
from week8_buggy import get_total, is_over_limit, format_summary

def test_get_total_correct():
    recs = [{'amount': 100}, {'amount': 200}]
    assert get_total(recs) == 300

def test_is_over_limit_true():
    assert is_over_limit(1500) is True

def test_is_over_limit_false():
    assert is_over_limit(500) is False

def test_is_over_limit_boundary():
    assert is_over_limit(1000) is False

def test_format_summary():
    result = format_summary(1234.56, 3)
    assert '1234.56' in result
    assert '3' in result
```

Line-by-line explanation:

- `from week8_buggy import get_total, is_over_limit, format_summary` — say explicitly, worth a brief note: this imports directly from the (now-fixed) `week8_buggy.py` file — the filename still says "buggy" even though the bugs are gone, since renaming it isn't part of today's task; not a problem, just worth acknowledging if a student finds the name slightly odd at this point.
- `test_get_total_correct` — a **fresh, independent test list**, not reusing the module-level `records` — a good habit worth naming: tests define their own isolated input data rather than

depending on whatever happens to exist elsewhere in the file, so a test's pass/fail never depends on unrelated code changing that data.

- `test_is_over_limit_boundary` — **the required boundary case**, exactly Week 7's lesson applied again: `is_over_limit(1000)` should be `False`, since the (now-fixed) condition is `amount > 1000`, strictly greater. Ask the room directly: "if Bug 2's fix had accidentally used `>=` instead of `>`, which test would catch that?" (This one specifically — a direct callback confirming the boundary-testing habit is becoming automatic, not just remembered from one specific prior lab.)
- `test_format_summary` — uses `in` to check that specific substrings appear in the result, rather than an exact `==` match against the whole formatted string — worth a brief note on why: this is more robust to minor formatting details (spacing, punctuation) that aren't the actual thing being tested, while still confirming the two genuinely important pieces of information (the dollar amount, the count) are present.

Run it:

```
pytest -v
```

Verified output — all five pass:

```
tests/test_week8.py::test_get_total_correct PASSED
tests/test_week8.py::test_is_over_limit_true PASSED
tests/test_week8.py::test_is_over_limit_false PASSED
tests/test_week8.py::test_is_over_limit_boundary PASSED
tests/test_week8.py::test_format_summary PASSED
5 passed
```

Common student mistakes to watch for:

- Forgetting that `week8_buggy.py` has module-level code (`total = get_total(records)`, `print(...)`) that runs automatically the moment it's imported — a curious student may notice the script's own `print()` output appearing when `pytest` runs, even though the test file itself never calls `print()`. Worth a brief, honest note: `pytest` captures output from *passing* tests by default (so it's not visible unless a test fails or `-s` is passed), but the underlying cause — importing a file with top-level executable code causes that code to run — is worth naming, since it's a real reason Week 7's "keep business logic files free of module-level side effects" discipline matters even more once tests start importing those files directly.

Check for understanding: "Which of these five tests would have caught Bug 1 (the `KeyError`), if it had run *before* the fix?" (`test_get_total_correct` — since it exercises `get_total()` directly, on data with correctly-spelled keys, it wouldn't itself have *shown* the typo bug (which was in the original `records` list's data, not in `get_total()`'s logic) — a good, slightly tricky question worth discussing: this test verifies `get_total()`'s *logic* is correct, but a typo in a *different* list literal, like the original buggy `records`, is a data-entry mistake this specific test was never designed to catch. Getting a student to articulate that distinction — logic correctness vs. data correctness — is a

genuinely valuable, subtle takeaway.)

6.6 Part 5 — AI Reflection and Push (0:55–1:07, 12 min)

Teaching goal: Complete `debug_log.md`'s AI Literacy Reflection honestly, and run the now-familiar ritual — closing the loop on today's central "Debug First, Then Ask" discipline.

Say to the class:

> *"Every section of `debug_log.md` needs to be finished now, honestly — including the AI reflection. If you didn't use AI today, say so, and say why you didn't need to. If you did, at any point, name the exact prompt and be specific about what you changed yourself afterward, in your own words."*

Do: Have every student finish `debug_log.md`'s remaining sections:

```
## After Fixing Both Bugs
Final output: Total: $5150.00 across 3 records / True
Did pytest pass? Yes

## AI Literacy Reflection
Did I use AI? Yes / No
If yes -- exact prompt I used: ___
What AI explained: ___
What I changed myself: ___
Can I explain every fixed line? Yes / No

## What I learned
[2–3 sentences]
```

Facilitation notes on the reflection specifically:

- **"Can I explain every fixed line? Yes / No"** — this is worth treating as the section's real test, worth reading aloud to the class: a student who can't honestly answer "Yes" here has a gap worth closing *now*, in class, while help is available — not discovering it during the Week 9 midterm.
- If a student used AI, "what AI explained" and "what I changed myself" should be **genuinely different content**, not the same thing restated twice — if a student's two answers read identically, that's worth a gentle individual check-in: did the AI's explanation actually inform an independent change, or was its output used essentially verbatim?
- "What I learned" — encourage something specific to *today's two bugs*, not a generic "debugging is important" — a strong answer names the actual distinction between the two error categories encountered (missing/mistyped data vs. mismatched types) or the value of the print-debugging/duck-explaining steps specifically.

Now the full ritual:

```
ruff format . && ruff check . && pytest
git add . && git commit –m 'lab 8: debugging ai literacy' && git push
```

Common student mistakes to watch for:

- Leaving `debug_log.md` sections blank or with placeholder underscores still in place — a quick visual scan of the file before committing catches this; the lab's explicit grading criteria include *all* sections completed, not just the code fixes.
- An AI Use Statement that's technically true but vague ("used AI a little") rather than specific (an actual prompt, an actual described change) — hold the same standard as every prior module's AI disclosure requirement: specific and honest, not just present.

Check for understanding: "If a classmate read only your `debug_log.md`, with no access to your code, could they understand what went wrong and how you found it?" (This is the actual bar the log is meant to meet — a good closing question, since it reframes the log from "paperwork to complete" to "a genuine record of your reasoning," which is the entire point of the exercise.)

6.7 Stretch — Write a Third Bug Yourself (1:07–1:11, as time allows)

Frame as a genuinely valuable inversion of today's exercise if the room reaches it:

> *"Introduce a third, deliberate bug into a copy of the script yourself — then document and fix it using the exact same workflow: traceback, duck, log, fix. Writing a bug on purpose and then debugging it is a surprisingly different skill from debugging one you stumbled into by accident — it requires understanding precisely what makes code fail, not just recognizing failure after the fact."*

A verified, illustrative example, if you want a ready one on hand rather than improvising live: change `format_summary`'s format spec from `{total:.2f}` to `{total:.2f%}` (an invalid format spec) — running it raises `ValueError: Invalid format specifier '.2f%' for object of type 'float'` — a good third error *category* (neither a `KeyError` nor a `TypeError`), reinforcing that the diagnostic process (read, duck, document, fix) generalizes across error types the student hasn't specifically been walked through today.

7 Wrap-Up (last ~4 minutes)

Review the submission checklist together:

- ☐ Git commit made, with a message including "lab 8"
- ☐ `week8_buggy.py` runs cleanly, both bugs fixed
- ☐ `tests/test_week8.py` contains all five tests, all passing
- ☐ `debug_log.md` fully completed, including an honest AI Literacy Reflection
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 9: "Next week is the midterm. Everything from Weeks 1 through 8 — the shell, venvs, Git, variables, conditionals, loops, dicts, functions, and today's debugging discipline — is fair game. If anything from today, or from the last three weeks, still feels shaky, this is the week to ask, not the night before."

8 Appendix A — Full Answer Key (`week8_buggy.py`, fixed + `tests/test_week8.py` + `debug_log.md` template)

```python
# week8_buggy.py -- both bugs fixed

records = [
    {'id': 1, 'name': 'Taylor', 'amount': 1200},   # fixed: was 'amout'
    {'id': 2, 'name': 'Jordan', 'amount': 450},
    {'id': 3, 'name': 'Morgan', 'amount': 3500},
]

def get_total(records):
    total = 0
    for rec in records:
        total += rec['amount']
    return total

def is_over_limit(amount):
    return amount > 1000   # fixed: '1000' (string) -> 1000 (int)

def format_summary(total, count):
    return f'Total: ${total:.2f} across {count} records'

total = get_total(records)
print(format_summary(total, len(records)))
print(is_over_limit(total))
```

Verified final output:

```
Total: $5150.00 across 3 records
True
```

```python
# tests/test_week8.py
from week8_buggy import get_total, is_over_limit, format_summary

def test_get_total_correct():
    recs = [{'amount': 100}, {'amount': 200}]
    assert get_total(recs) == 300

def test_is_over_limit_true():
    assert is_over_limit(1500) is True

def test_is_over_limit_false():
    assert is_over_limit(500) is False

def test_is_over_limit_boundary():
    assert is_over_limit(1000) is False

def test_format_summary():
    result = format_summary(1234.56, 3)
    assert '1234.56' in result
    assert '3' in result
```

`debug_log.md` — full template, filled with model answers for Bug 1/2:

```
# debug_log.md
# Author: [Your Name]

## Bug 1
Error type: KeyError
File + line: week8_buggy.py, line 10 (inside get_total)
What the error means: code tried to access a dictionary key
  ('amount') that doesn't exist in one of the records
What I told the duck: [student's own words]
What I tried: added a DEBUG print inside get_total() to see each
  record's actual contents before the key access that was failing
How I fixed it: corrected the typo 'amout' to 'amount' in the first
  record's dictionary; removed the DEBUG print afterward

## Bug 2
```

```
Error type: TypeError
File + line: week8_buggy.py, line 14 (inside is_over_limit)
What the error means: Python cannot compare an int and a string
  with > -- the two sides of the comparison are different types
How I fixed it: removed the quotes around 1000 so is_over_limit
  compares two integers instead of an int and a string

## After Fixing Both Bugs
Final output: Total: $5150.00 across 3 records / True
Did pytest pass? Yes

## AI Literacy Reflection
Did I use AI? [honest answer]
If yes -- exact prompt I used: [specific]
What AI explained: [specific]
What I changed myself: [specific]
Can I explain every fixed line? Yes

## What I learned
[2–3 genuine sentences]
```

9 Appendix B — Extra Practice (only if the class finishes early)

Five required parts fill the full class period at a normal pace, especially given the deliberate, unhurried pace of Parts 1–3. If a section moves unusually fast:

Extra — a fourth test, for the fix's robustness. Have students add `test_get_total_empty`, asserting `get_total([]) == 0` — an **empty-list boundary case**, a different kind of edge than a numeric threshold: what happens when there's simply nothing to accumulate. (Verified: passes — `total = 0` initialized correctly handles a loop that never executes.)

Extra — narrate a *working* function to the duck. Have students pick any already-correct function from Week 7's `business_rules.py` and rubber-duck explain it line by line, the same way Part 1 did for the broken `get_total()` — a good reminder that rubber duck debugging is useful even *without* a bug present, as a way of confirming genuine understanding versus surface-level familiarity.